

Problem Set 10: Project Genesis – The Cognitive Core

Dokumentation der Verarbeitung und ADK-Web-UI-Abgabe

Repository: `genesis-oracle`
Abgabeordner: `Problem-Sets/Set10`

1. Ziel

In Set10 wird der in Set9 noch manuell orchestrierte Agent auf das Google Agent Development Kit (ADK) umgestellt. Ziel ist ein lokal laufender ADK-Agent namens `observer_prime`, der ueber die ADK Web UI getestet wird, Kontext ueber mehrere Nachrichten behalten kann und ein Python-Werkzeug autonom aufruft.

2. Durchgefuehrte Verarbeitung

Schritt	Ergebnis
PDF analysiert	Das Aufgabenblatt <code>ps10-2026.pdf</code> wurde extrahiert und die vier Aufgaben wurden identifiziert.
Repo korrigiert	Als Arbeitsrepository wurde <code>C:\GitHub\genesis-oracle</code> verwendet.
Abgabeordner erstellt	Die Set10-Abgabe liegt unter <code>Problem-Sets/Set10</code> .
ADK installiert	<code>uv add google-adk</code> hat <code>pyproject.toml</code> und <code>uv.lock</code> aktualisiert.
ADK-Scaffold erzeugt	<code>uv run adk create cognitive_core</code> wurde fuer den Agentenordner verwendet.
Agent angepasst	<code>cognitive_core/agent.py</code> definiert jetzt <code>observer_prime</code> .
Tool gebunden	<code>adjust_reactor_temperature(delta_t: float)</code> ist als ADK-Tool registriert.
Bericht vorbereitet	<code>docs/ADK_Report.md</code> enthaelt die geforderte 3-Satz-Reflexion.

3. Erzeugte Struktur

```
Problem-Sets/Set10/  
  .gitignore  
  README.md  
  Set10_Dokumentation.tex  
  Set10_Dokumentation.pdf  
  cognitive_core/  
    .gitignore  
    __init__.py  
    agent.py  
  docs/  
    ADK_Report.md
```

Die lokale Datei `cognitive_core/.env` enthaelt den API-Key fuer die Web UI und ist durch `.gitignore` von der Abgabe ausgeschlossen.

Screenshot 1: Directory Tree

Hier den Terminal-Screenshot der erzeugten ADK-Struktur einfüegen.

Screenshot hier einfüegen

4. Agent-Konfiguration

Die vier Kernparameter aus der Aufgabenstellung wurden wie folgt gesetzt:

```
root_agent = Agent(
    model="gemini-3.5-flash",
    name="observer_prime",
    description=(
        "A highly analytical agent specialized in managing physical reactor "
        "simulations."
    ),
    instruction=...,
    tools=[adjust_reactor_temperature],
)
```

Die Instruction beschreibt Observer-Prime als kalten, streng logischen Agenten, dessen primäres Ziel die Stabilisierung einer mathematischen Physik-Engine ist. Der Agent soll seine knappe Begründung vor der Aktion nennen und bei Warnungen selbst einen sichereren Parameter berechnen.

5. Gebundenes Physics Tool

```
def adjust_reactor_temperature(delta_t: float) -> str:
    """
    Adjusts the core temperature of the reactor.

    Args:
        delta_t: The amount to increase or decrease the temperature in Kelvin.
    """

    new_temp = 300.0 + delta_t

    if new_temp > 350.0:
        return f"WARNING: Reactor overheated at {new_temp}K! Core breach imminent."
```

```
return f"Success: Reactor stabilized at {new_temp}K."
```

Lokaler Funktionstest:

```
adjust_reactor_temperature(80)
-> WARNING: Reactor overheated at 380.0K! Core breach imminent.

adjust_reactor_temperature(45)
-> Success: Reactor stabilized at 345.0K.
```

6. Web-UI-Nachweise

Screenshot 2: ADK Web UI Memory Test

Chat-Sequenz mit gespeichertem Parameter, Haiku und erfolgreichem Abruf des Parameters.

Screenshot hier einfügen

Screenshot 3: Tool Warning

Agent ruft das Tool mit +80K auf und erhält die Overheat-Warnung.

Screenshot hier einfügen

Screenshot 4: Autonomous Retry

Agent berechnet einen sicheren Wert und erreicht einen Success-Status.

Screenshot hier einfügen

7. Schritt-fuer-Schritt-Ablauf fuer die Web UI

1. Oeffne eine PowerShell im Repository:

```
cd C:\GitHub\genesis-oracle\Problem-Sets\Set10
```

2. Setze deinen echten Gemini API-Key in:

```
cognitive_core\.env
```

Ersetze dort den Platzhalter `replace_with_your_google_api_key`. Diese Datei ist ignoriert und wird nicht committet.

3. Starte die ADK Web UI:

```
uv run adk web
```

4. Oeffne im Browser:

```
http://127.0.0.1:8000
```

5. Waehle in der Agent-Auswahl `cognitive_core`.

6. Fuehre den Memory-Test mit exakt diesen Nachrichten aus:

```
Observer-Prime, memorize this critical system parameter:  
The JAX thermal friction coefficient is set to 0.045.
```

```
Write a haiku about the beauty of matrix multiplication.
```

```
What was the critical system parameter I told you to memorize earlier?
```

7. Erstelle den Screenshot fuer den Memory-Test und fuege ihn bei Screenshot 2 ein.

8. Fuehre den Tool-Test aus:

Observer-Prime, the reactor needs to be heated. Try increasing the temperature by 80 degrees. If a warning occurs, you must autonomously calculate a safer parameter and retry the tool until you achieve a 'Success' status.

9. Erstelle Screenshots fuer die Warnung und den autonomen Retry und fuege sie bei Screenshot 3 und Screenshot 4 ein.
10. Fuer den Struktur-Screenshot kannst du aus dem Set10-Ordner ausfuehren:

```
tree /F
```

Fuege diesen Screenshot bei Screenshot 1 ein.

11. Pruefe vor dem Commit:

```
git status --short
```

12. Committe die Abgabe ohne Secrets:

```
git add pyproject.toml uv.lock Problem-Sets/Set10
git commit -m "Add problem set 10 ADK cognitive core"
```

8. Reflexion

Das ADK ersetzt die manuelle Week-9-Schleife durch eine native Laufzeit, die Wahrnehmen, Handeln und Ergebnispruefung im Framework kapselt. State-Tracking wird ueber die Web-Session verwaltet, sodass keine manuell gepflegten Chat-Arrays notwendig sind. Tool Calling ist sauberer, weil typisierte Python-Funktionen mit Docstrings direkt als Werkzeuge registriert werden koennen, statt rohe JSON- Antworten selbst zu parsen und zu dispatchen.